

# FPGA experiment report

Task: FPGA 1

Student 1 name: Saleh Khalil ID: 213310485

Student 2 name: Mahmood Stitia ID: 214032682

## Part 1 – Status update

Fill in the following table according to your progress at the time of the submission.

Each row should refer to one of the modules or a simulation. For example, write FA under module name and refer to FA.v and FA\_tb.v in that line.

The status column should be filled with one of the following:

Done – If both are written and simulation passed.

Partially done – If you started writing the module and simulation but it's not working properly.

Not started – If you didn't get to that part of the task.

When the status is *partially done*, please use the notes/comments column to explain in a few more words what is the status of the module and what is the status of the simulation.

| Module name/Step | status | notes/comments |
|------------------|--------|----------------|
| FA               | Done   |                |
| CSA              | Done   |                |

Fill here:

### Issues:

In this section please describe the current issues you are facing.

Per each module and simulation, if you receive any errors at the time of submission, please describe which errors and what they mean.

Fill here:

FA: None

CSA: None

## Part 2 – Simulations and answers

In this section, please add the images for each simulation as described in the report. Remember to use annotations and explain in your own words what is shown in switches in each simulation.

In some of the tasks in the report, there are also questions that you should answer here.

Make sure to include images of all simulations running at the time of submission, even if they are not correct.

Fill here:

FA

```
1 module FA(a, b, ci, sum, co);
2
3   input  a, b, ci;
4   output sum, co;
5   wire axb;
6
7   // FILL HERE
8   assign axb = a^b;           // a xor b
9   assign sum = axb^ci;         // a xor b xor ci
10  //calculate carry out
11  assign co = (a&b) | (axb)&ci ;
12
13 endmodule
```

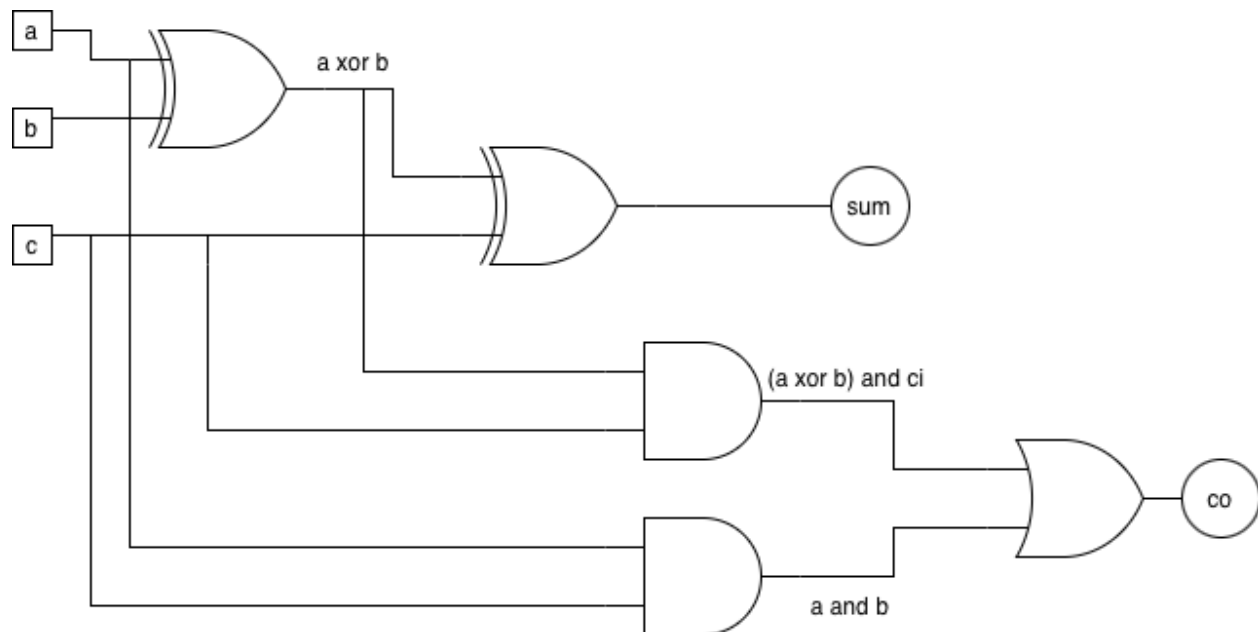


Figure 1: Block diagram of the Full Adder

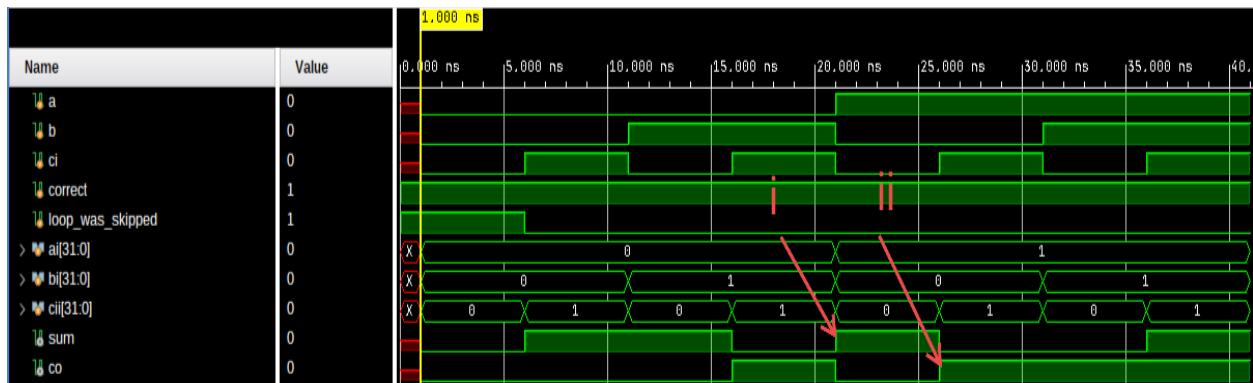


Figure 2: Detailed view of the Full Adder simulation.

The first line represents the value of “a”. The second line represents the value of “b”. The third line represents the value of “ci” . The fourth and fifth lines represent the values of “correct” and “loop\_was\_skipped”.

We use these two values to verify whether our calculation is correct. The bottom two lines represent the value of “sum” and the value of “co” (carry-out) respectively; these values are the final outputs from our design where:

$$\text{sum} = a \oplus b \oplus c_{in}$$

$$c_{out} = (a \cdot b) + (b \cdot c_{in}) + (a \cdot c_{in})$$

- **i:** The first red arrow points to the results of the transition at 20 ns. At this moment, the inputs switch to "a" with value 1, "b" with value 0, and "ci" with value 0. the output of "sum" changes from 0 to 1 to accurately reflect this result.
- **ii:** The second red arrow illustrates the change at 25 ns when “ci” goes high (from 0 to 1). With “a” still holding 1 and “b” holding 0, Because the decimal number 2 is represented as binary 10, the “sum” drops back to 0. The arrow points down to show that “co” changes from 0 to 1 at that exact same time to output the carry.

## CSA

```
1 module CSA(a, b, ci, sum, co);
2
3     parameter N=4;
4     parameter K = N >> 1;
5
6     input [N-1:0] a;
7     input [N-1:0] b;
8     input ci;
9     output [N-1:0] sum;
10    output co;
11
12
13    // FILL HERE
14    generate
15        if (N!=1) begin
16            wire [K-1:0] s_r;
17            wire carry_r;
18            wire [N-K-1:0] s_l0;
19            wire carry_l0;
20            wire [N-K-1:0] s_l1;
21            wire carry_l1;
22            //right case (alice)
23            CSA #(K) csa_r(a[K-1:0],b[K-1:0],ci,s_r,carry_r);
24            //left case (bob)
25            CSA #(N-K) csa_l0(a[N-1:K],b[N-1:K],1'b0,s_l0,carry_l0);
26            CSA #(N-K) csa_l1(a[N-1:K],b[N-1:K],1'b1,s_l1,carry_l1);
27            //choose the correct case
28            assign sum[K-1:0] = s_r;
29            assign sum[N-1:K] = (carry_r==1'b1)?s_l1:s_l0;
30            assign co = (carry_r==1'b1)?carry_l1:carry_l0;
31        end else begin ;
32            FA fa(a,b,ci,sum,co);
33        end
34    endgenerate
35
36
37 endmodule
```

## CSA Simulation Result

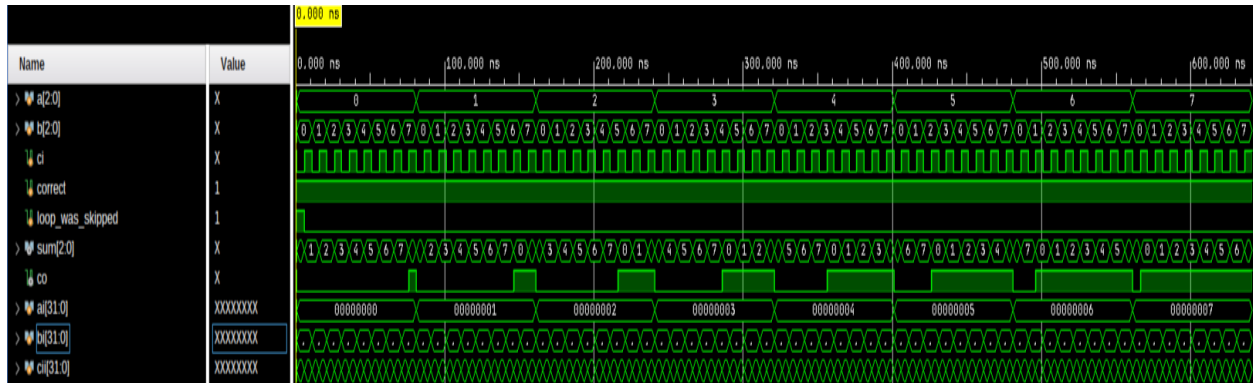


Figure 3: Overall view of the Carry Adder simulation.

The first line represents the 3-bit value of “a”. The second line represents the 3-bit value of “b”. The third line represents the value of “ci” (carry-in). The fourth and fifth lines represent the values of “correct” and “loop\_was\_skipped”, respectively. We use these two values to verify whether our calculation is correct. The sixth and the seventh lines represent the 3-bit value of “sum” and the 1-bit value of “co” (carry-out) respectively; these values are the outputs from our design where:



Figure 4: Zoomed in view of the simulation focusing on carry propagation.

In the waveform viewer we can see the value of each parameter at each time, and how the outputs react to the inputs. For example:

- **i:** The first red arrow points to the exact moment (at 565 ns) when the value of “ci” changes from 0 to 1. If you look at the inputs at this moment, “a” holds value 7 and “b” holds value 0.
- **ii:** The second red arrow illustrates the immediate result of “ci” going high. Because 8 is too large to fit in a 3-bit wire, the “sum” rolls over to 0. The arrow points downward to show that “co” changes from 0 to 1 at exactly the same time to output the carry.